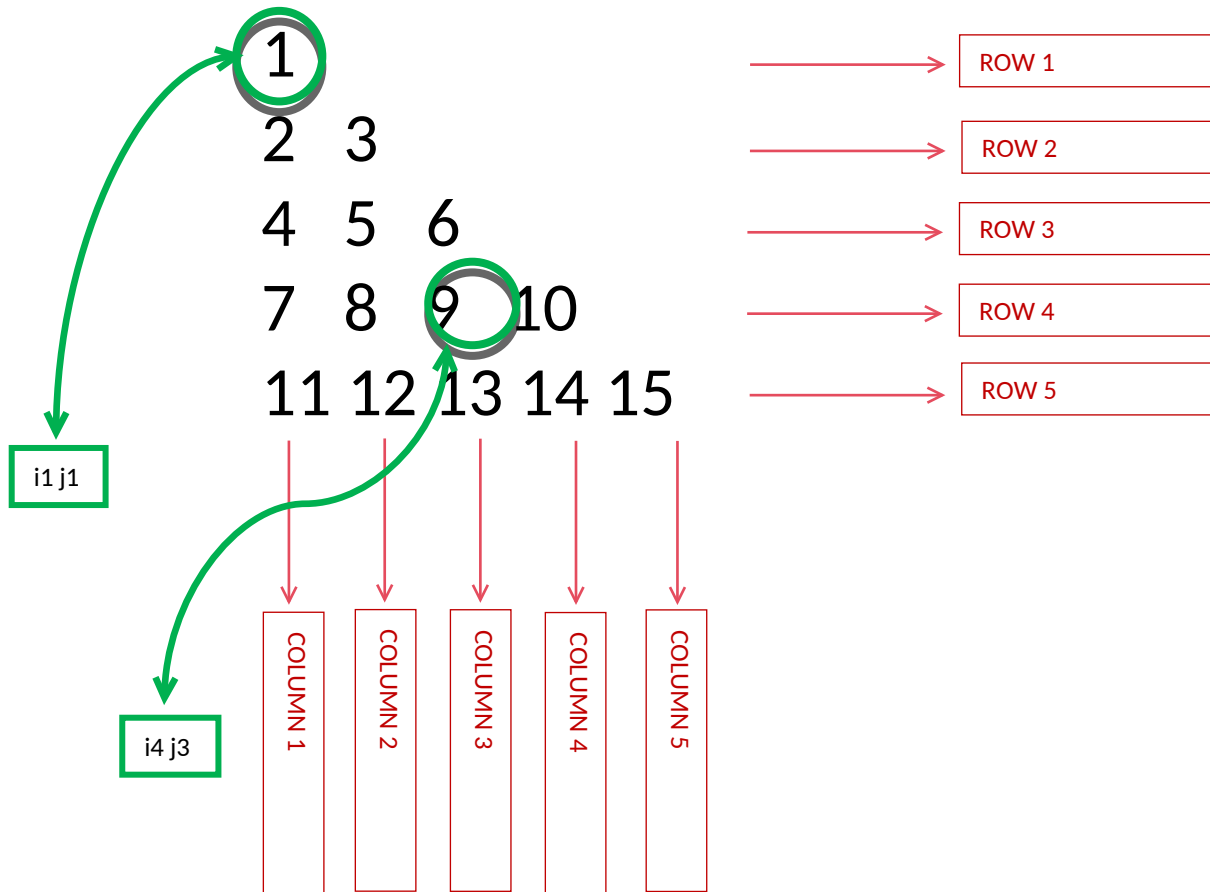


WEEK 01 MODULE
Programming Practice

TOPIC : 2 'Number Pattern'

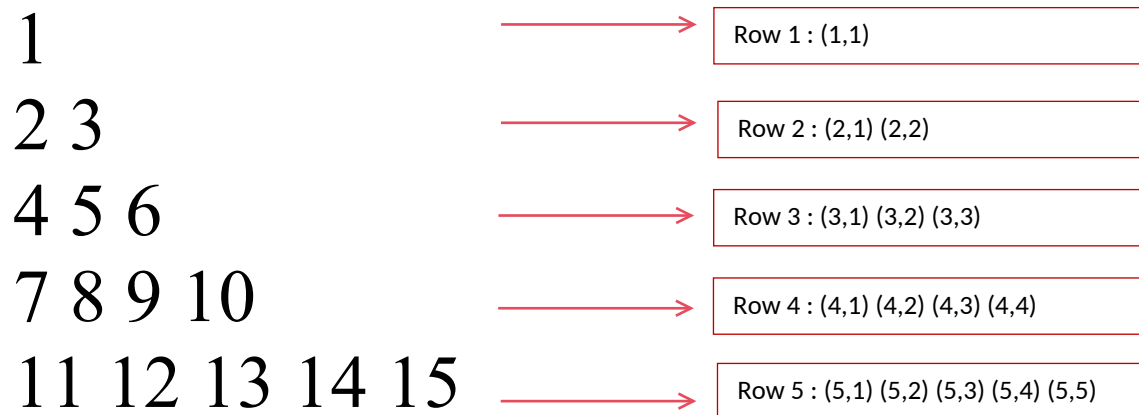
3. CONTINUOUS COLUMN PATTERN



1. Problem Analysis :-

i. Pattern Structure:

- The output forms a **right-angled continuous number triangle shape**.
- This pattern is based on the **Right Angled Triangle structure**.
- Unlike star patterns, this pattern uses continuously increasing **numbers** instead of symbols.
- The number of elements increases row by row.
- For n=5



i. Execution Flow:

Step 1: Observe the Grid Concept:

- The pattern can be visualized as a triangular portion of an $n \times n$ grid. Each row prints only a subset of the available columns.

- size = $n \times n$.

- Each cell in the grid represents a unique coordinate (i, j), where i denotes the row index and j denotes the column index.

- For $n = 5$:

Observation:

- Row 1 $\rightarrow$ 1
- Row 2 $\rightarrow$ 2 3
- Row 3 $\rightarrow$ 4 5 6
- Row 4 $\rightarrow$ 7 8 9 10
- Row 5 $\rightarrow$ 11 12 13 14 15

A clear relationship can be observed:

- Number of elements per row = i
- Values do NOT reset per row
- A single variable maintains continuity across entire pattern

Step 2: Traverse the grid row by row using outer loop.

- Create a variable before that to store the next number to print

- The outer loop is responsible for generating rows.

- count = 1

- for (i = 1 to n)

Step 3: For every row, traverse columns .

- for (j = 1 to i)
- control the number of the elements in that row.

Step 4: Print logic:

inside inner loop

- Print current value of count
- Then increment count++

Step 5: After all numbers of the current row are printed, move to the next line.

Step 6: Repeat the same process until all rows have been printed and counter reaches $n(n+1)/2$ elements.

- The pattern growth is controlled by the inner loop limit.
- the inner loop runs from 1 to the current row number:

$j \leq i$ (This automatically increases the number printed in each row.)

{

for(j = 1; j <= i; j++)

Meaning:

Current row gets as many number as its row number.

}

2. Program :-

```
package week1_module;
import java.util.Scanner;

public class NumberTriangle {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of rows: ");
        int n = sc.nextInt();

        for(int i = 1; i <= n; i++) {

            for(int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }

            System.out.println();
        }

        sc.close();
    }
}
```

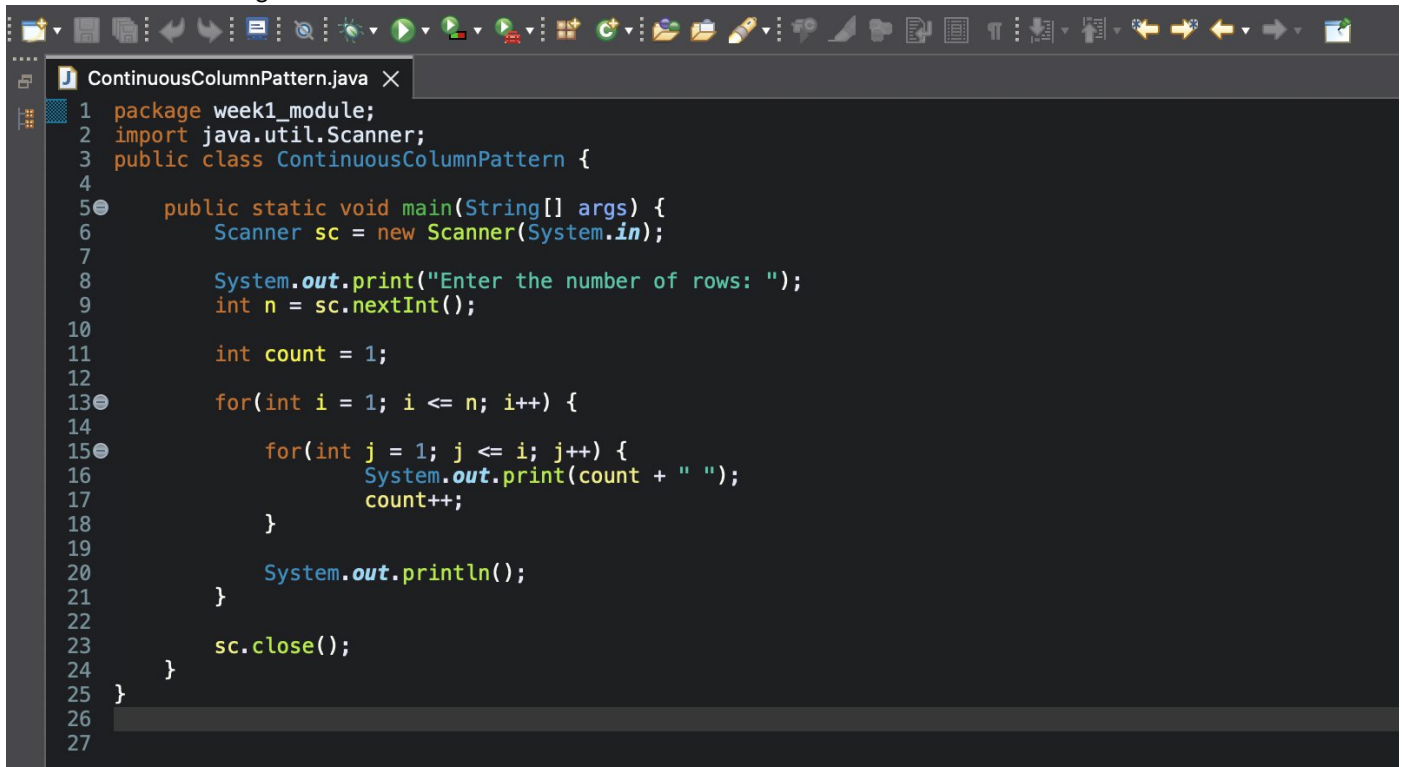
3. Program Output :-

Enter the size of the rows: 5

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

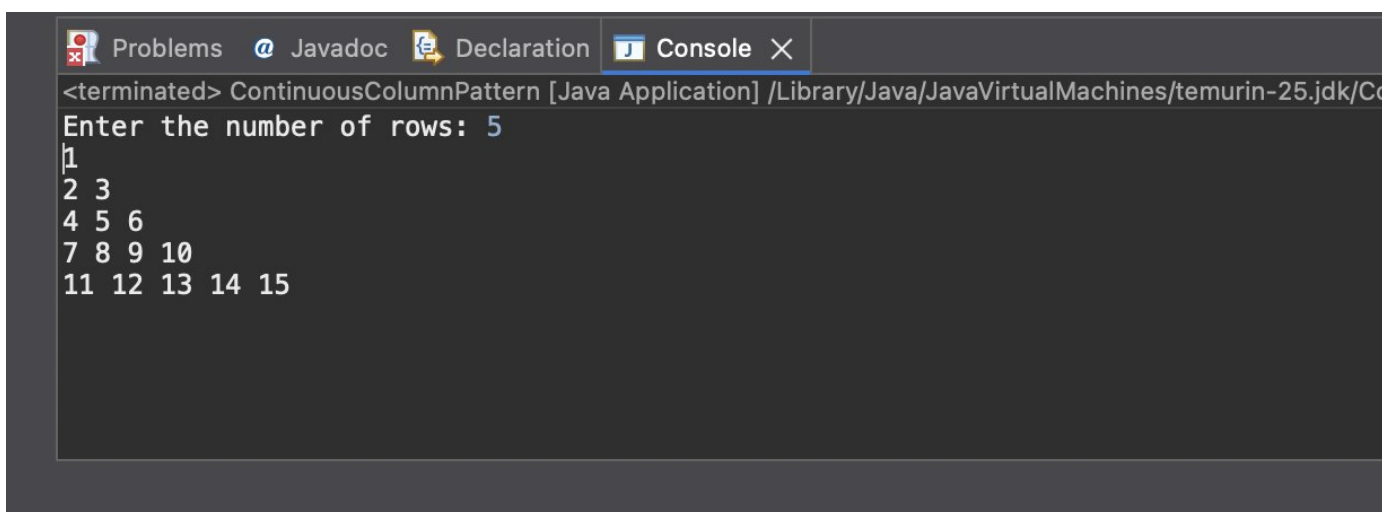
4. ScreenShot :-

i. Program :-

A screenshot of an IDE window titled 'ContinuousColumnPattern.java'. The code is as follows:

```
1 package week1_module;
2 import java.util.Scanner;
3 public class ContinuousColumnPattern {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         System.out.print("Enter the number of rows: ");
9         int n = sc.nextInt();
10
11         int count = 1;
12
13         for(int i = 1; i <= n; i++) {
14
15             for(int j = 1; j <= i; j++) {
16                 System.out.print(count + " ");
17                 count++;
18             }
19
20             System.out.println();
21         }
22
23         sc.close();
24     }
25 }
26
27
```

ii. Program's Output :-

A screenshot of the IDE's console window. The title bar shows 'Problems', 'Javadoc', 'Declaration', and 'Console'. The console output is as follows:

```
<terminated> ContinuousColumnPattern [Java Application] /Library/Java/JavaVirtualMachines/temurin-25.jdk/C
Enter the number of rows: 5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```